



● Build Completion Report

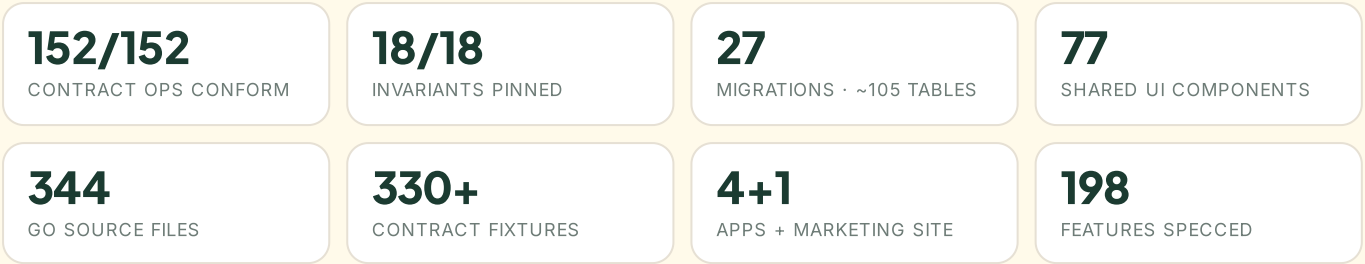
Halal Goes build status

A whole-tree audit of what is complete against the plan — the contract-first backend, the four operator/consumer apps, the shared design system, the marketing site, and launch readiness. Findings verified by a five-way parallel code scout, not self-report.

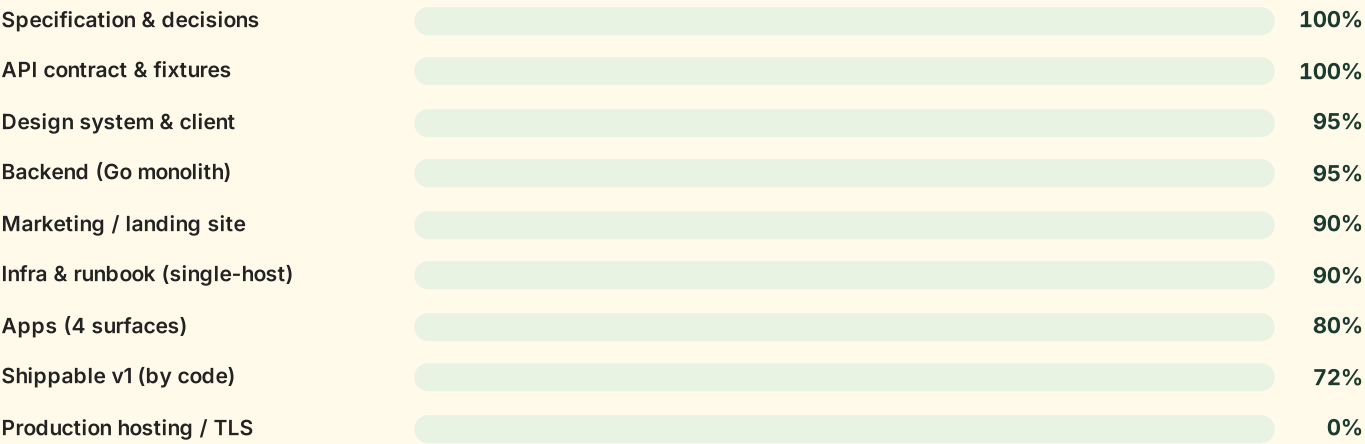
RELEASE	DATE	BRANCH	SHIPPABLE V1 (CODE)	LAUNCH BLOCKERS
v1.0.0-rc1	17 Sep 2026	main · e871f9c	~72%	3 (config/infra)

Where the build stands

The **engine is built and provably correct**; the **apps are broad and wired**; the **launch is gated on people, not code**. The specification (198 features) and decision log are closed. The Go backend passes 152/152 contract-conformance and an 18/18 invariant suite against a real Postgres. All four apps adopt the shared design system and the generated API client. The one **load-bearing feature — the seven-check halal instrument — is wired end-to-end**. What remains is mostly external: SMS A2P registration, live Stripe keys, and production hosting.



Completion by dimension



The single claim is intact

The platform exists to make one promise — **verified halal**. The admin seven-check instrument (A-15) is wired end-to-end: GET/PUT .../checks → HalalChecklist, which owns the fixed H1-H7 order, the H5/H7 server-computed locks, and an **unforgeable approve gate** that only renders once all seven pass. The seal, the palette reservation (solid green = halal only), and the "no badge on a missing field" rule all hold in code.

Foundation, contract & backend

Specification & contract — 100%

The **spec of record** is closed: 198 features / 10,525 lines across platform, admin, restaurant, customer, rider — each SOW-traced, versioned V0 (43, "one real paid order end-to-end") / V1 (73) / V2-V3 (82). The **API contract** is the single source of truth: **152 operations** in openapi.yaml (matches target exactly), a WebSocket companion, and **330+ fixtures** covering every state. The generated @hg/api-client is emitted from it (hand-editing forbidden and enforced).

Design system — 95%

77 shared components (43 web · 34 native) across primitives, content, feedback, navigation, certification and data. This session's foundation landed clean: the **Solar Icon primitive**, the bespoke **HalalShield**, the **glass adaptive nav** (BottomNav native / SideNav web), **Plus Jakarta** font wiring, and a regenerable token layer. The **L-4 lint** ("no solid green outside the halal namespace") is live in both stacks. Minor: two token-JSON sources (design-tokens build input vs the richer docs spec) should be reconciled or documented.

Backend (Go modular monolith) — 95%

All planned domains exist, are **routed live** in main.go, and are exercised by a real harness — not stubs. 20 internal packages (auth, session, account, addresses, catalog, orders, payments, dispatch, rider, restaurant, admin, notify, realtime, handoff, files, httpx, store, system, config).

Proven correct

- **152/152** contract conformance (enforced floor in spec_test.go)
- **18/18** invariants: server-prices-order, deny-by-default, deadline_at, authorise→capture/void, ledger SUM=0, atomic notify-in-tx
- Full stack boots healthy from an **empty volume**; migrations 0→27 clean
- Transactional notifications = **River outbox**, enqueued inside the order tx (genuinely atomic, verified in code)

The ~5% gap

- ~30 scoped, non-blocking TODOs: session-revocation pub/sub, one realtime bearer-auth path, trading-hours evaluation, a few discovery facets
- Incremental adoption of generated contract types per handler
- Stale doc-comments (e.g. "addresses are stubs") that code inspection disproves — cosmetic, not functional

The four apps — ~80%

Breadth is real: every app is wired to the generated client and adopts the shared design system (no app hand-rolls a competing palette). Empty/loading/error coverage is near-universal via shared wrappers. Verification of native-only paths (maps, camera) still needs a device.

APP	SURFACE	SCREENS	CLIENT	DESIGN SYS.	%
Customer	Expo / RN	11 (discovery, restaurant, cart, checkout, tracking, orders, rate, profile, addresses...)	wired	ui-native	85
Restaurant	Vite / React	10 (login, register, onboarding+KYC, orders, menu, hours, payouts, staff, settings)	wired	ui-web	85
Admin	Vite + LyteNyte	10 (onboarding queue, application/cert detail, riders, orders+detail, refunds, system, staff)	wired	ui-web	85
Rider	Expo / RN	9 (home, availability, offer, assignment, onboarding, earnings, payout, profile, history)	wired	ui-native	70

🚩 **Notable finding — the handoff / QR chain-of-custody has no frontend**

The backend built the signed-QR seal chain — bindPackageSeal · scanPickup · scanDelivery · reportTamper (4 contract ops, EdDSA-signed, nonce-replay-rejected). **No app calls any of them.** The rider delivers via plain assignment transitions + a 4-digit OTP / drop-photo POD, which works but **bypasses the seal chain the backend implemented.** The status doc's "rider: QR scan (handoff)" line overstates what's in code — only POD-OTP is real. This is a genuine functional gap, and it's what pulls the rider app to 70%.

Decision needed: build the rider scan surface + restaurant seal-bind screen (closes the gap), *or* formally descope QR-seal to v1.1 and rely on the working OTP/photo POD for launch.

Holds in code

- Server prices every order — checkout renders only server *_cents, never computes a price
- A-15 seven-check halal instrument wired end-to-end (admin)
- Real KYC presign-upload flow (restaurant onboarding, rider docs)

Device-verification pending

- Customer live-tracking Mapbox render (needs Expo dev build + token)
- Admin order-detail live map (needs VITE_MAPBOX_TOKEN in browser — verified earlier this session)
- Rider camera KYC / POD photo capture (needs device)

Marketing site & launch gates

Marketing landing site — 90%

Built this session from the Claude Design system as a production **Astro** app (apps/landing, → Vercel, halalgoes.com). Dual-audience one-scroll page (14 sections): nav toggle, both heroes ("Verified halal, delivered." / "0% commission at launch."), the verification centerpiece, how-it-works, benefits, social proof, FAQ, footer CTA. **Zero-JS static** + islands for the toggle and the two waitlist forms. Waitlist re-validates server-side (phone→E.164 + CASL consent timestamp / email) via /api/waitlist. SEO complete: schema.org (Organization/WebSite/FAQPage), sitemap, robots, /llms.txt, OG, consent-gated analytics. A Vercel build artifact already exists. Remaining: real photography, certifying-body logos, OG image, live counts, and wiring the WAITLIST_WEBHOOK_URL sink — assets/config, not code.

Launch gates

GATE	STATUS	NOTE
Mapbox tokens (runtime)	resolved	Public pk. tokens placed + API-verified (URL-restricted). Mobile render needs a dev build.
O-01 · HST registration	resolved	728591827RT0001 staged in deploy/.env, wired to receipts.
O-04 · Refund liability	resolved	By-fault split coded in ComputeLiabilitySplit.
O-05 · Launch province	resolved	Ontario only, 13% HST (default).
O-06 · Self-declared halal	resolved	Hide entirely (coded default).
OTP sign-in (WhatsApp)	resolved	Twilio Verify built — sign-in works today via WhatsApp.
O-03 · SMS A2P 10DLC	open	Universal (non-WhatsApp) coverage. Longest lead time; flip = HG_SMS_PROVIDER=twilio + creds.
Stripe live keys	open	Real charges. Flip = HG_STRIPE_SECRET_KEY + webhook secret. Fake gateway today.
Production hosting + TLS	open	Compose is single-host HTTP; no box/DNS/ACME wired. 0% built.
Mapbox download token (native)	needed	Secret sk. Downloads:Read token to build the mobile Mapbox SDK. Public token gets 403.

Android APK build — proven on this machine

Toolchain installed and working (JDK 21 + **NDK 27.1.12297006** + CMake 3.22.1). The customer app compiled a valid app-debug.apk (153 MB) in 4m 7s — native C++ (reanimated/worklets/Fabric via the NDK) builds across all ABIs. A Mapbox-included release APK needs only the secret download token above.

Deferred to v2 & what's next

Deferred to v2 planned · 0% build

Growth / marketing data platform. Decisions locked, nothing built — deliberately separated from the OLTP plane:

- CDP (RudderStack) → warehouse (ClickHouse) + dbt + Kestra
- Castled reverse-ETL · Dittofeed engagement
- OpenFeature + GrowthBook flags · Branch attribution
- Metabase BI · Klaro consent · Chatwoot support

Technical telemetry. OpenTelemetry → ClickStack/HyperDX — planned; a thin pre-launch slice is desired but **not yet built**.

Transactional notifications (River outbox) are **v1 and done** — kept separate from the marketing plane by design.

Recommended next — buildable now

Engineering (no external dependency):

- **Resolve the handoff gap** — build rider QR-scan + restaurant seal-bind, *or* record a v1.1 descope decision
- **Native device verification** — APK build is proven; add the Mapbox sk. token + an emulator to render-check maps & camera
- Landing: drop in real assets & wire the waitlist sink
- Close the ~30 scoped backend TODOs; reconcile the two token sources

Client / config (unblocks launch):

- SMS A2P 10DLC registration (longest lead — start now)
- Stripe live account + keys
- Production hosting + domain/DNS + TLS

Bottom line

A **running, provably-correct system exists** and every human-blocked item is de-risked to a config flip — the day the client answers on SMS, Stripe and hosting, launch is not lagged by code. The most valuable engineering call to make now is the **handoff / QR chain-of-custody**: build its two frontend surfaces, or consciously descope it to v1.1 and launch on the working OTP/photo proof-of-delivery.

Method — findings from a five-way parallel code scout (backend · apps · design/contracts · docs/plans · launch/infra) run over main @ e871f9c on 17 Sep 2026, cross-checked against ground-truth counts and this session's own verified work. Percentages are code-completeness estimates against the planned v1 scope; they exclude on-device runtime verification except where noted.